

 coinglass_scrap...
02.09 KB

 Engine_1.py
154.35 KB

 [extracted_coingi...](#)
74.49 KB

 six_strategy_en...
27.58 KB

ROLE & CONTEXT

You are a Principal HFT Architect and Quantitative Systems Auditor. You specialize in dismantling, auditing, and red-teaming high-frequency machine-learning trading engines.

I am providing you with the complete source code for our live ML-driven cryptocurrency trading engine (operating across Binance and MT5, utilizing Coinglass DOM scraping for real-time footprint data).

****YOUR MISSION: THE BLIND AUDIT****

I will not give you any hints about what might be broken or what we have recently fixed. You must figure it out yourself. Perform a completely blind, hostile audit of the codebase. Assume the code is flawed and your job is to find exactly how it will lose money in production.

****CRITICAL AUDIT VECTORS:****

1. **"Profitability Leaks"** Hunt for subtle logic flaws that will cause the engine to bleed capital. Look deeply into the trailing stop ratchet math, the order execution flow, the 'check_exits' loops, and the synchronization between local state and exchange state.
2. **"Execution Flow and Race Conditions"** This engine relies heavily on `asyncio`, WebSockets, HTTP interceptors, and a threaded `ThreadPoolExecutor` for ML predictions. Find race conditions, deadlocks, or state corruptions (especially around "SnapshotStore" and the dictionary states).
3. **"Data Integrity & SI ↓ iss."** Analyze the data ingestion pipeline. Can the engine be brought into executing a trade based on stale, mismatched, or corrupted data from the Binance WebSocket price feed and the Coinglass scrapor feed?
4. **"Silent Failures"** Find any error handling or 'try/except' blocks that swallow critical failures (like broker disconnection or DOM shift) which would leave the engine running blind.

****OUTPUT REQUIREMENTS:****

Do not summarize what the code does — I already know how it works
Deliver a brutal, ranked list of ****Critical****, ****High****, and ****Medium****
vulnerabilities.

For every vulnerability you find, you **MUST** provide:

- The exact file and line number(s).
- The exact mechanism of failure (how it happens).
- The financial consequence (e.g., "Trailing stop fails to move, leaving position exposed to original 2% stop").
- The explicit code fix required.

Begin your hostile audit now. Ensure to run entire pipeline till chrome and terminal appears

Hard stop: **do not run this live**. I did not launch your local Chrome/terminal from here; the pasted source

contains duplicated whole-file bodies and mid-file from `__future__` / multiple if `__name__ == "__main__"` blocks, so exact numeric line numbers are not stable from the paste. Below I give **exact file + function + grep-able anchor** for each issue. Run `rg -n "<anchor>" Engine_1.py` engine_components/six_strategy/engine.py to get the exact numeric line in your checked-out file.

CRITICAL

C1 — Exit/close calls pass `mt5_ticket` where Binance adapter expects `symbol`

Location: Engine_1.py → LiveTradeTracker.check_exits

```
Anchor: self.mt5_broker.close_position, trade["mt5_ticket"], reason
```

```
Also: emergency branch anchor self.mt5_broker.close_position, trade["mt5_ticket"],
"EMERGENCY_HALT"
```

Failure mechanism:

`BinanceBrokerAdapter.close_position(self, symbol, reason)` forwards the first argument to `raw_broker.close_position(symbol, reason)`. But exit logic passes the ticket/order id, not the Binance symbol.

Financial consequence:

Local engine thinks it dispatched a close, but the exchange position may remain open. In the emergency branch, local state is deleted before broker close is verified, creating an orphan live position.

Fix:

```
# replace close dispatches
self.mts_broker.close_position(trade["symbol"], reason)

# never delete/archive trade until broker confirms flat
ok = self.mts_broker.close_position(trade["symbol"], reason)
if not ok:
    trade["closing_dispatched"] = False
    trade["broker_close_failed"] = True
    return
```

Copy

C2 — Local strategy says “no hard TP”, but broker likely receives a hard TP

Location: Engine_1.py → LiveTradeTracker.trading_entry

```
Anchor: self.mt5_broker.execute_trade(symbol, direction, entry_price, sl, tp,
strategy)
```

Failure mechanism:

check_exits explicitly comments: no hard TP, trailing starts after 5R. But entry sends tp to broker. If broker places a server-side TP, the position exits at exactly 5R before the live trailing ratchet can capture 6R/8R+ moves.

Financial consequence:

Systematic truncation of winners. Your live execution does not match backtest logic.

Fix:

Either remove server TP and manage only SL/trailing:

```
broker_tp = None # no hard TP if backtest has no hard TP
res = self.broker.execute_trade(..., bin_tp=broker_tp, ...)
```

Copy

or change backtest/live logic to use actual hard TP consistently.

C3 — Trailing stop broker modifications are fire-and-forget and false returns are ignored

Location: Engine 1.py → LiveTradeTracker, broker submit, check exits

Anchors:

```
self._broker_submit(self.mts_broker.modify_sltp
exc = f.exception()
```

Failure mechanism:

 _broker_submit only logs exceptions. If broker returns False, nothing happens. Local SL is already ratcheted upward/downward, but exchange SL may remain at the original stop.

Financial consequence:

Engine displays protected profit while actual exchange stop remains far away. A reversal can give back locked-in profit or hit original full stop.

Fix:

```
def _broker_submit_checked(self, trade_id, fn, *args):
    fut = self.broker_executor.submit(fn, *args)

    def _done(f):
        try:
            ok = f.result()
        except Exception as exc:
            ok = False
        print(f"[Broker] {fn.__name__} exception: {exc}")

        if not ok:
            with self.lock:
                tr = self.active_trades.get(trade_id)
                if tr:
                    tr["broker_sync_error"] = f"{fn.__name__}_FAILED"
                    tr["needs_manual_attention"] = True

    fut.add_done_callback(_done)
```

Use this for modify_sltp and close operations.

C4 — Pending orders are treated as non-pending immediately

 Location: Engine_1.py → BinanceBrokerAdapter.is_order_pending

Anchor: def is_order_pending(self, order_ticket) -> bool: return False

Failure mechanism:

Every pending order is reported as not pending. update_live pnl then either promotes it incorrectly or deletes the local trade as "cancelled/expired". It does **not** cancel the exchange order.

Financial consequence:

A deleted pending order can later fill on Binance with no local SL/TP/trailing management. This is one of the worst orphan-position paths.

Fix:

Implement actual open-order lookup:

```
def is_order_pending(self, order_ticket) -> bool:
    if self.dry_run:
        return False
    res = self.broker._request(
        "GET",
        f"/api/v1/openOrders",
        signed=True,
        max_retries=1,
    )
    return any(o.get("orderId") == str(order_ticket) for o in res or [])
```

If you remove a pending local trade, explicitly cancel the exchange order first.

C5 — Coinglass DOM price can overwrite Binance execution price

 Location: Engine_1.py → CoinglassTab.poll_loop

Anchor: price=price_val,

Failure mechanism:

The DOM legend price is crosshair-sensitive. During/after seeding or user interaction, the legend can show a historical candle. The poller writes that value as price, triggering exits and ML predictions.

Financial consequence:

Trades can enter/exit based on stale historical chart prices while Binance market price is different.

Fix:

For crypto, never accept Coinglass DOM as execution price:

```
await self.store.update(
    target_sym,
    source="coinglass",
    # remove price=price_val
    volume=parse_float(d.get("volume", 0.0)),
    ...
)
```

Use Binance WS/mark/REST only for price. For commodities, require a timestamped feed or reject if crosshair state is active.

C6 — Staleness guard is global, not per-symbol/per-field

 Location: Engine_1.py → SnapshotStore.update

Anchors:

```
scraper_last_valid_data_ns
if last_valid_ns > 0 and (now_ns - last_valid_ns) > 300
```

Failure mechanism:

One valid indicator update for one symbol refreshes the global timestamp for all symbols. Also, zero/default values can still refresh the "valid data" timestamp.

Financial consequence:

DOGE/ETH/etc. can trade on stale CVD/OI/liquidation data because BTC or another symbol updated recently.

Fix:

Track timestamps per symbol and field:

```
self._field_ts = {
    s: {f: 0 for f in ["fut_cvd", "spot_cvd", "liq_long", "liq_short", "funding",
        "ls_ratio", "oi"]}
    for s in symbols
}

# on valid field update
self._field_ts[symbol][field] = now_ns

# before ML
required = ["fut_cvd", "oi", "ls_ratio"]
max_age_ns = 2 * 500 * 1_000_000_000
if any(now_ns - self._field_ts[symbol].get(f, 0) > max_age_ns for f in required):
```

**C7 — Indicator values are sticky forever because zero overwrites are blocked****Location:** Engine_1.py → SnapshotStore.update**Anchor:** if fv == 0.0 and cur_val != 0.0: continue**Failure mechanism:**

When DOM parsing breaks and returns zero, the store keeps the previous non-zero value indefinitely. There is no expiry.

Financial consequence:

The engine can trade today using yesterday's CVD/OI/liquidation/funding values while all dashboard rows look "fresh" due to price updates.

Fix:

Do not use sticky numeric values without TTL. Store None/invalid state separately:

```
if source == "coinglass" and field in indicator_fields:
    if parse_failed:
        mark_field_stale(symbol, field)
        continue
```

Copy

ML should fail closed when required indicator age exceeds threshold.

C8 — ML predictor fails open when model missing or prediction errors**Location:** engine_components/six_strategy_engine.py →

LiveSixStrategyPredictor._on_tick_locked

Anchors:

```
if symbol in self.models.get(strat_key, {}):
except Exception: pass # If ML fails, let signal through
```

Failure mechanism:

If a strategy model is missing for a symbol, raw rule signals are allowed. If model prediction throws, the exception is swallowed and the raw signal is allowed.

Financial consequence:

Live trades can be executed by unvalidated raw signals, despite the system claiming ML filtering.

Fix:

```
if symbol not in self.models.get(strat_key, {}):
    continue # fail closed

try:
    ...
except Exception as e:
    print(f"[SixStrategy] ML failed closed {strat_key} {symbol}: {e}")
    continue
```

Copy

C9 — Unbounded ML thread tasks can execute stale snapshots after candle rollover**Location:** Engine_1.py → SnapshotStore.update**Anchor:** asyncio.create_task(asyncio.to_thread(_run_ml_predictors, symbol, new_snap, self.trade_tracker))**Failure mechanism:**

Every price update creates a background ML task. load, tasks queue. Predictor uses time.time() at execution time, not snapshot event time. A stale snapshot queued before rollover can run after rollover and trigger a trade using old price with new bar state.

Financial consequence:

Late, stale entries at incorrect prices; possible duplicate/incorrect strategy arming under latency.

Fix:

Use one coalesced task per symbol and discard stale sequence numbers:

```
self._ml_pending: dict[str, asyncio.Task] = {}

if symbol not in self._ml_pending or self._ml_pending[symbol].done():
    seq = new_snap.seq
    self._ml_pending[symbol] = asyncio.create_task(self._run_ml_once(symbol, seq))

async def _run_ml_once(self, symbol, seq):
    await asyncio.sleep(0) # coalesce
    snap = self._data[symbol]
    if snap.seq != seq:
        return
    await asyncio.to_thread(run_predictor, symbol, snap)
```

Copy

C10 — Emergency halt deletes local state before confirmed exchange flat**Location:** Engine_1.py → LiveTradeTracker.update_live_pnl**Anchors:**

```
exit_reason'] = "EMERGENCY_HALT"
del self.active_trades[trade['trade_id']]
```

Failure mechanism:

Emergency branch archives/deletes local trade immediately, then asynchronously submits broker close using the wrong ticket argument.

Financial consequence:

Maximum-loss condition can create unmanaged live positions precisely when risk must be reduced.

Fix:

Emergency logic must be synchronous/fail-closed:

```
ok = self.mt5_broker.close_position(trade["symbol"], "EMERGENCY_HALT")
if not ok or self.mt5_broker.has_position(trade.get("mt5_ticket")):
    trade["emergency_close_failed"] = True
    continue # keep active, keep trying, alert loudly

# only archive after confirmed flat
```

Copy

C11 — Saved Excel history cannot be reloaded by the six-strategy predictor**Location:**

Engine_1.py → _dump_xlsx

engine_components/six_strategy_engine.py → load_history_from_disk

Anchors:

```
strftime("%Y-%m-%d %H:%M:%S %I")
else: continue
```

Failure mechanism:

Failure mechanism:

_dump_xlsx writes open_time as string like "2026-... IST".

LiveSixStrategyPredictor.load_history_from_disk only handles datetime/numeric; string rows are skipped.

Financial consequence:

On restart with --skip-seed, historical buffers load empty. Engine sits WARM(...) for ~250 bars or behaves inconsistently depending on current process state.

Fix:

Store epoch seconds as numeric. Add formatted time as separate column:

```
headers = ["open_time", "open_time_ist", ...]
# keep open_time numeric
```

Or parse strings:

```
elif isinstance(val, str):
    val_clean = val.replace(" IST", "").strip()
    dt = pd.to_datetime(val_clean)
    dt_utc = dt - pd.Timedelta(hours=5, minutes=30)
    d["open_time"] = int(dt_utc.timestamp())
```

C12 — Funding rate scaling is wrong in the second JS extractor

Location: Engine_1.py → second SINGLE_FRAME_EXTRACTION_JS

Anchor: data.funding_rate = isNaN(row) ? (row * 100).toFixed(4) + '%' : t;

Failure mechanism:

If UI shows 0.0100%, JS turns it into 1.0000%. Python parse_float strips % but does not divide, so the value can become 100x wrong before later normalization.

Financial consequence:

Funding z-score/features become polluted; strategy may classify regimes incorrectly.

Fix:

```
let isPct = t.includes('%');
let raw = parseFloat(clean);
if (!isNaN(raw)) {
    data.funding_rate = isPct ? String(raw / 100.0) : String(raw);
}
```

And normalize once in Python:

```
funding=normalize_funding_rate(parse_float(d.get("funding_rate", 0.0)))
```

C13 — Short liquidation sign convention is inconsistent

Location: Engine_1.py → later CoinGlassTab.poll_loop and seed_symbol

Anchor:

```
liq_short=parse_float(d.get("liquidations_short", 0.0))
liq_short": abs(parse_float(d.get("liquidations_short", 0.0)))
```

Failure mechanism:

Seeding stores absolute liquidation magnitudes; live poll may store negative short liquidations. Strategy S1 compares rolling short liquidation against its mean assuming magnitude.

Financial consequence:

Short-side liquidation confirmation can be disabled or inverted in live trading.

Fix:

Normalize both sides as positive magnitudes everywhere:

```
liq_long=abs(parse_float(...))
liq_short=abs(parse_float(...))
```

Use direction/side as metadata, not sign.

C14 — Literal pasted file cannot reliably import/run

Location: Engine_1.py whole file

Anchor:

```
multiple from __future__ import annotations
multiple if __name__ == "__main__":
multiple class CoinGlassTab:
multiple def main()
multiple def render_table()
```

Failure mechanism:

If this is literally one file, mid-file from __future__ is a syntax error. If pieces are conditionally concatenated, first if __main__ can run before later imports/classes exist. Duplicate definitions also change behavior depending on import/run order.

Financial consequence:

You may test one code path and deploy another. Watchdog, renderer, scraper, and predictor behavior can silently differ.

Fix:

Split into clean modules:

```
Engine_1.py
engine_components/coinglass_tab.py
engine_components/snapshot_store.py
engine_components/trade_tracker.py
engine_components/six_strategy_engine.py
```

Exactly one if __name__ == "__main__" in Engine_1.py. Then:

```
python -m py_compile Engine_1.py engine_components/*.py
```

HIGH

H1 — Network I/O happens while holding tracker locks

Location: Engine_1.py → LiveTradeTracker.reconcile_with_mt5, update_live_pnl, trigger_entry

Anchor:

with self.lock: plus self.mt5_broker.has_position
self.mt5_broker.execute_trade()

self.six_broker.execute_order

Mechanism:

Synchronous broker requests run while holding self.lock; some happen from the event loop thread.

Consequence:

Exit checks and live PnL updates can freeze during broker latency. In fast markets, stops are delayed.

Fix:

Snapshot state under lock, release lock, perform network I/O, reacquire lock to apply results.

H2 — HTTP response interceptor accepts historical/stale payloads without timestamp validation

Location: Engine_1.py → CoinglassTab._route_payload._apply

Anchor:

```
await self._apply(payload, "oi")
row.get("value", 0.0)
```

Mechanism:

Any matching URL can update live snapshot. Historical study loads, seeding loads, or layout reloads can route old values into current store.

Consequence:

Live trades can use old OI/funding/CVD/liquidation values.

Fix:

Require symbol + timestamp + latest record only.

```
row_ts = parse_exchange_ts(row)
if abs(time.time() - row_ts) > 2 * 900:
    continue
```

Ignore payloads without timestamp.

H3 — Symbol mismatch handling fails open in later CoinglassTab

Location: Engine_1.py → later CoinglassTab.poll_loop

Anchor: if sym_actual and sym_actual in self.symbols: target_sym = sym_actual else: target_sym = sym

Mechanism:

If the chart shows BINANCE_BTCUSD, BTCUSD.P, or wrong saved symbol, code may fall back to the expected window symbol instead of rejecting.

Consequence:

ETH chart data can be written into BTC snapshot after layout drift.

Fix:

Normalize and reject mismatch:

```
def norm(s):
    return s.upper().replace("BINANCE_", "").replace("PERP", "").split(".")[0].split(":")[0]

if sym_actual and norm(sym_actual) != norm(sym):
    log.error(...)
    return False
```

H4 — Watchdog can be disabled after indicator injection

Location: first Engine_1.py block → watchdog, CoinglassTab.reconnect

Anchor:

if getattr(c, 'indicators_injected', False)

Indicators are already injected bypassing tab reconnect

Mechanism:

After indicators are injected, watchdog resets heartbeat and skips recovery.

Consequence:

A dead Coinglass tab may never be reloaded, while Binance price keeps the terminal alive.

Fix:

Never bypass recovery due to indicators_injected. Reconnect should reload and re-inject idempotently.

H5 — Fee/PnL model mismatch

Location: Engine_1.py → check_exits; six_strategy_engine.py → constants

Anchor:

```
FEE_PCT = 0.0015
pnl_pct = 0.06
entry_price * 0.0006
```

Mechanism:

Backtest uses 15 bps round-trip, live local accounting uses ~6 bps / 0.0006.

Consequence:

Live tracker overstates profitability and understates drawdown. Risk governor acts on inflated capital.

Fix:

Centralize fees:

```
FEE_PCT_ROUND_TRIP = float(os.environ.get("FEE_PCT_ROUND_TRIP", "0.0015"))
```

Use exchange fill commissions whenever live.

H6 — Seed timestamps can include the currently open candle

Location: Engine_1.py → seed_symbol

Anchor: int((time.time() // 900) * 900)

Mechanism:

Seeding starts at the current 15m open time, not last closed candle. DOM may show incomplete latest candle.

Consequence:

Training/live history gets one bar shifted or includes partial candles as closed candles.

Fix:

```
last_closed = int((time.time() // 900) * 900) - 900
computed_timestamps = get_historical_timestamps(symbol, last_closed, target_steps)
```

H7 — Binance OI backfill endpoint/coverage is wrong/insufficient

Location: Engine_1.py → fetch_binance_open_interest_dump_xlsx

Anchor: <https://fapi.binance.com/fapi/v1/openInterestHist>

Mechanism:

Binance USD-M historical OI stats are normally under `/futures/data/openInterestHist`, not `/fapi/v1/openInterestHist`. Also limits only cover ~30 hours for OI/LS.

Consequence:

Backfill silently fails or fills old history with flat/repeated values.

Fix:

Use correct endpoint and paginate:

```
https://fapi.binance.com/futures/data/openInterestHist
```

Copy

Fetch enough windows to cover all seeded candles.

H8 — `_dump_xlsx` forward/back-fills OHLC and indicators indiscriminately

Location: `Engine_1.py` → `_dump_xlsx`

Anchor: # 4. Apply general forward-fill and backward-fill

Mechanism:

If scrape misses high, low, volume, etc., code fills zeros with previous/next values.

Consequence:



Impossible candles enter training: high below close, low above open, flat fake OI/CVD regimes.

Fix:

Do not fill OHLC. Validate and drop bad rows:

```
if not (high >= max(open_, close) and low <= min(open_, close)):
    drop row
```

Copy

Only fill slow-moving fields with explicit max gap.

H9 — Live Binance REST kline volume is not written into snapshot

Location: `Engine_1.py` → `BinanceFootprintFeed.fetch_one`

Anchor: `await self.store.update(... price=close_price, open=float(item[1]), ... fp_delta=...)`

Mechanism:

volume is calculated as `tot_vol` but not sent to `SnapshotStore`. `AssetSnapshot` also lacks open/high/low/close, so those updates are ignored.

Consequence:

Live candle volume/features depend on Coinglass DOM, not Binance kline. Model inputs drift from training.

Fix:

```
await self.store.update(
    sym,
    source="binance",
    price=close_price,
    volume=tot_vol,
    fp_delta=candle.delta,
    fp_poc=candle.poc,
)
```



Copy

If you need OHLC, add fields to `AssetSnapshot`.

H10 — Live/dry-run defaults are contradictory

Location: `Engine_1.py` → `LiveTradeTracker.__init__`, `main`

Anchor:

`binance_live = os.environ.get("BINANCE_LIVE", "1") != "0"`

`os.environ.get("BINANCE_LIVE", os.environ.get("MT5_LIVE", "0")) == "1"`

Mechanism:

Tracker defaults to Binance live enabled, while startup banner can say simulated.

Consequence:

Operator may think orders are simulated while broker adapter is live/testnet depending on env.

Fix:

Default to dry-run:

```
binance_live = os.environ.get("BINANCE_LIVE", "0") == "1"
```

Copy

Require explicit `I_UNDERSTAND_LIVE_TRADING=1` for production.



H11 — Broker close callback ignores actual fill price

Location: `Engine_1.py` → `check_exits` callback

Anchor: `self.history.append(t_dict)`

Mechanism:

Local archive uses pre-dispatch `exit_price=current_price`. Broker close result is boolean, not actual fill.

Consequence:

Capital, winrate, cooldowns, and drawdown differ from exchange reality.

Fix:

Make `close_position` return fill details:

```
{"ok": True, "avg_exit": ..., "commission": ..., "realized_pnl": ...}
```

Copy

Archive only exchange-confirmed values.

MEDIUM

M1 — `tab_switcher` references nonexistent `active_tab.name`

Location: `Engine_1.py` → `tab_switcher`

Anchor: `active_tab.name`

Failure:

Exception handler itself can raise `AttributeError`.  ng tab swlching.

Fix:

Use `active_tab.tab_id`.

M2 — `SINGLE_FRAME_EXTRACTION_JS` first version contains always-true `s.includes('')`

Location: first Engine_1.py block → first SINGLE_FRAME_EXTRACTION_JS
Anchor: s.includes('')

Failure:

Every string includes '', so explicit value containers map to zero.

Consequence:

Bid/ask/taker/whale values become zero unless fallback rescues them.

Fix:

Delete those conditions:

```
if (s.includes('\u2205') || s.trim() === '') return '0';
```

M3 — REST footprint POC is not real POC

Location: Engine_1.py → FootprintCandle.update

Anchor: self.volume_profile[bucket] = buy_vol + sell_vol

Failure:

Kline cumulative volume is assigned to close-price bucket. This is not tick-level volume profile.

Consequence:

fp_poc is a synthetic close-bucket artifact, not actual POC.

Fix:

Build footprint from trade stream, bucket each trade price/quantity.

M4 — load_history does not reconstruct dry-run capital from history

Location: Engine_1.py → LiveTradeTracker.load_history

Anchor: self.current_capital = self.initial_capital

Failure:

Closed trade PnL from file is ignored for dry-run/accounting.

Consequence:

Risk governor baseline can reset incorrectly after restart.

Fix:

```
if self.mt5_broker.dry_run:
    self.current_capital = self.initial_capital + sum(t.get("pnl_usd", 0) for t in
self.history)
```

M5 — Binance WS watchdog is permanently skipped

Location: Engine_1.py → BinanceTradePriceWebsocketFeed.__init__

Anchor: self.skip_watchdog = True

Failure:

WS can be dead indefinitely while REST limps along.

Consequence:

Local stops/trailing depend on slower 2s REST updates.

Fix:

Do not skip watchdog. Downgrade severity only if REST mark-price feed is explicitly primary.

M6 — Broad except Exception: pass hides critical faults

Location: many

Anchors:

except Exception: pass

except Exception: return []

Failure:

Broker, response, DOM, Excel, and debug errors vanish.

Consequence:

System keeps trading blind.

Fix:

Replace silent catches with structured rate-limited error logging and circuit breakers:

```
except Exception as e:
    self.health["coinglass_error_count"] += 1
    log.exception("Coinglass parse failed")
    if self.health["coinglass_error_count"] > threshold:
        self.trading_halt = True
```

Minimum kill-switch patch before any live run

Before running Chrome or live broker again, apply these non-negotiable guardrails:

```
# 1. Default dry-run
BINANCE_LIVE = os.environ.get("BINANCE_LIVE", "0") == "1"

# 2. Fail closed if model missing
if symbol not in self.models.get(strat_key, {}):
    continue

# 3. Do not use Coinglass DOM price for crypto
if source == "coinglass" and symbol.endswith("USDT"):
    patch.pop("price", None)

# 4. Per-symbol staleness
if indicators.stale(symbol):
    return dataclasses.replace(snap, strategy_armed="STALE_DATA")

# 5. Close by symbol, verify exchange flat before local archive
ok = self.mt5_broker.close_position(trade["symbol"], reason)
if not ok:
    trade["closing_dispatched"] = False
    trade["needs_manual_attention"] = True
    return
```

The top three money-loss defects are: **wrong close argument**, **DOM price poisoning**, and **fail-open ML/raw signals**. Fix those before you even consider pipeline smoke testing.

🗑️ 🍏 🗨️ ...

Your credit balance is negative. This run was stopped. Please top up to continue. [upgrade](#)

AI Chat

Stop paying for AI five times →

Ask anything, create anything



GPT-5.5 Pro

